EXPERIMENT -4

AIM- Using the Employee schema, write queries with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare execution plans using EXPLAIN.

OBJECTIVES

1. To understand and implement different types of SQL joins.
2. To retrieve related data using INNER JOIN and LEFT JOIN.
3. To understand and implement a self-join on the Employee table.
4. To perform a 3-way join between Employee, Department, and Project tables.
5. To understand and implement correlated subqueries.
6. To use the EXISTS operator for checking the existence of related records.
7. To simulate the functionality of INTERSECT and EXCEPT using SQL queries.
8. To analyze and compare the execution plans of SQL queries using EXPLAIN.

THEORY

INNER JOIN

An INNER JOIN returns only those records that have matching values in both tables based on the specified join condition. It is used to combine related information from two or more tables.

For example, the Employee and Department tables can be joined using the common DeptID attribute to display employees along with their departments.

LEFT JOIN

A LEFT JOIN returns all records from the left table and the matching records from the right table. If no matching record exists in the right table, NULL values are returned for the columns of the right table.

It is useful when all records from the primary or left-side table must be retained, even if they do not have corresponding records in the other table.

SELF-JOIN

A self-join is a join in which a table is joined with itself. Different aliases are assigned to the same table so that its records can be compared with one another.

It can be used to find relationships between employees, such as employees working in the same department or employees having similar salaries.

3-WAY JOIN

A 3-way join combines three tables in a single query using related attributes.

In the Employee schema, the Employee table can be joined with both Department and Project tables. This allows information such as employee name, department name, and project name to be retrieved together.

CORRELATED SUBQUERY

A correlated subquery is a subquery that depends on a value from the outer query. Unlike an independent subquery, it is logically evaluated for each row considered by the outer query.

Correlated subqueries are useful when the result of the inner query needs to be compared with the current row of the outer query, such as finding employees whose salary is greater than the average salary of their own department.

EXISTS

The EXISTS operator checks whether a subquery returns at least one row.

It returns TRUE when the subquery produces one or more matching records and FALSE when no matching record exists. It is commonly used to determine whether a related record exists in another table.

SIMULATED INTERSECT-

INTERSECT returns records that are common to the results of two queries.

When INTERSECT is not supported by a particular SQL system, its functionality can be simulated using alternatives such as INNER JOIN or EXISTS. The columns and data types involved in the comparison should be compatible.

SIMULATED EXCEPT-

EXCEPT returns records that are present in the first query result but absent from the second query result.

When EXCEPT is not supported, it can be simulated using NOT EXISTS or an appropriate LEFT JOIN with a NULL check

EXPLAIN

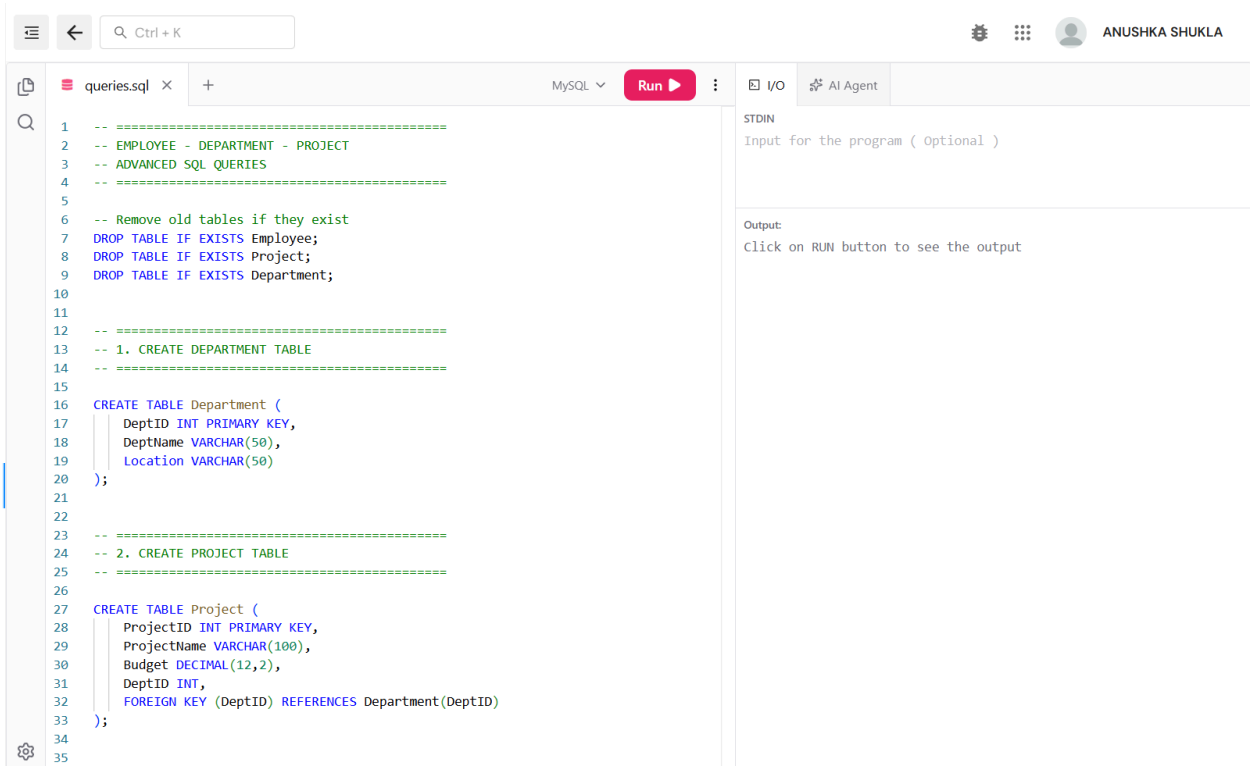
EXPLAIN is used to display the execution plan that the database optimizer intends to use for a SQL query.

It provides information about how tables will be accessed and joined, such as:

- Table scans
- Index usage
- Join methods
- Estimated number of rows
- Filtering operations
- Query execution cost

Comparing execution plans helps in understanding query performance and identifying inefficient queries. An execution plan with fewer scanned rows or an appropriate index access path may indicate a more efficient execution strategy, although the exact interpretation depends on the database system and data distribution.

SOURCE CODE




```sql
-- ============================================
-- EMPLOYEE - DEPARTMENT - PROJECT
-- ADVANCED SQL QUERIES
-- ============================================

-- Remove old tables if they exist
DROP TABLE IF EXISTS Employee;
DROP TABLE IF EXISTS Project;
DROP TABLE IF EXISTS Department;

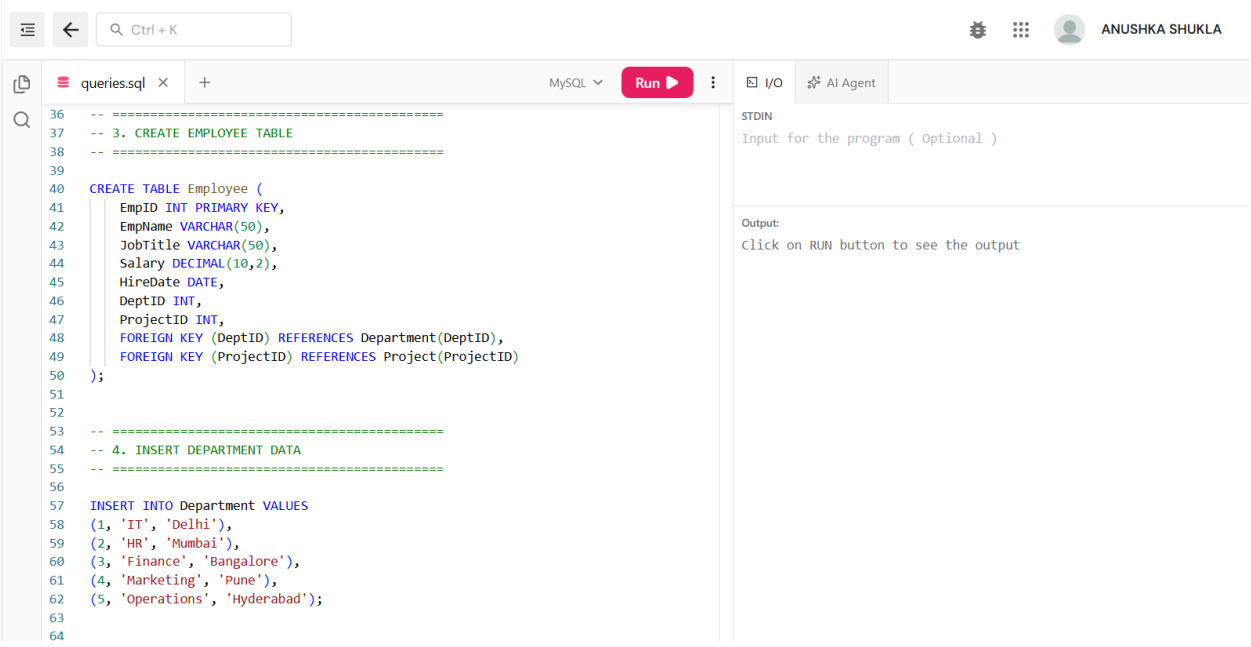

-- ============================================
-- 1. CREATE DEPARTMENT TABLE
-- ============================================

CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50),
    Location VARCHAR(50)
);


-- ============================================
-- 2. CREATE PROJECT TABLE
-- ============================================

CREATE TABLE Project (
    ProjectID INT PRIMARY KEY,
    ProjectName VARCHAR(100),
    Budget DECIMAL(12,2),
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

```



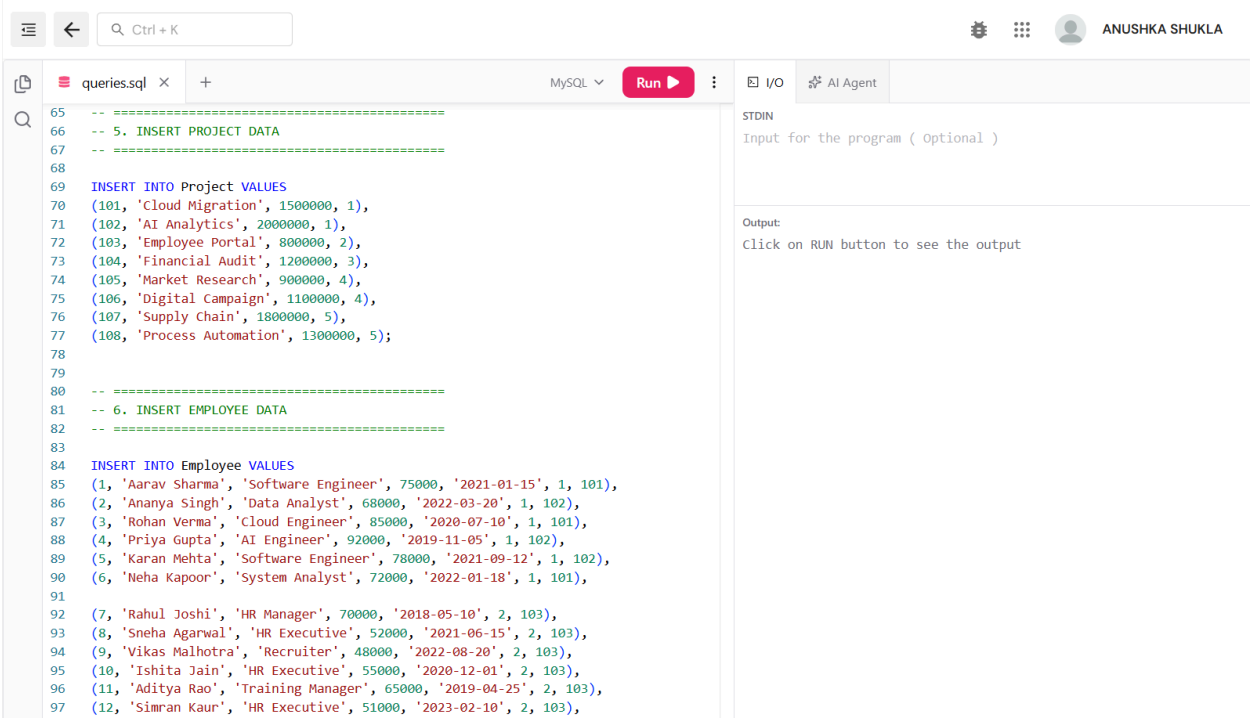

```sql
-- ============================================
-- 3. CREATE EMPLOYEE TABLE
-- ============================================

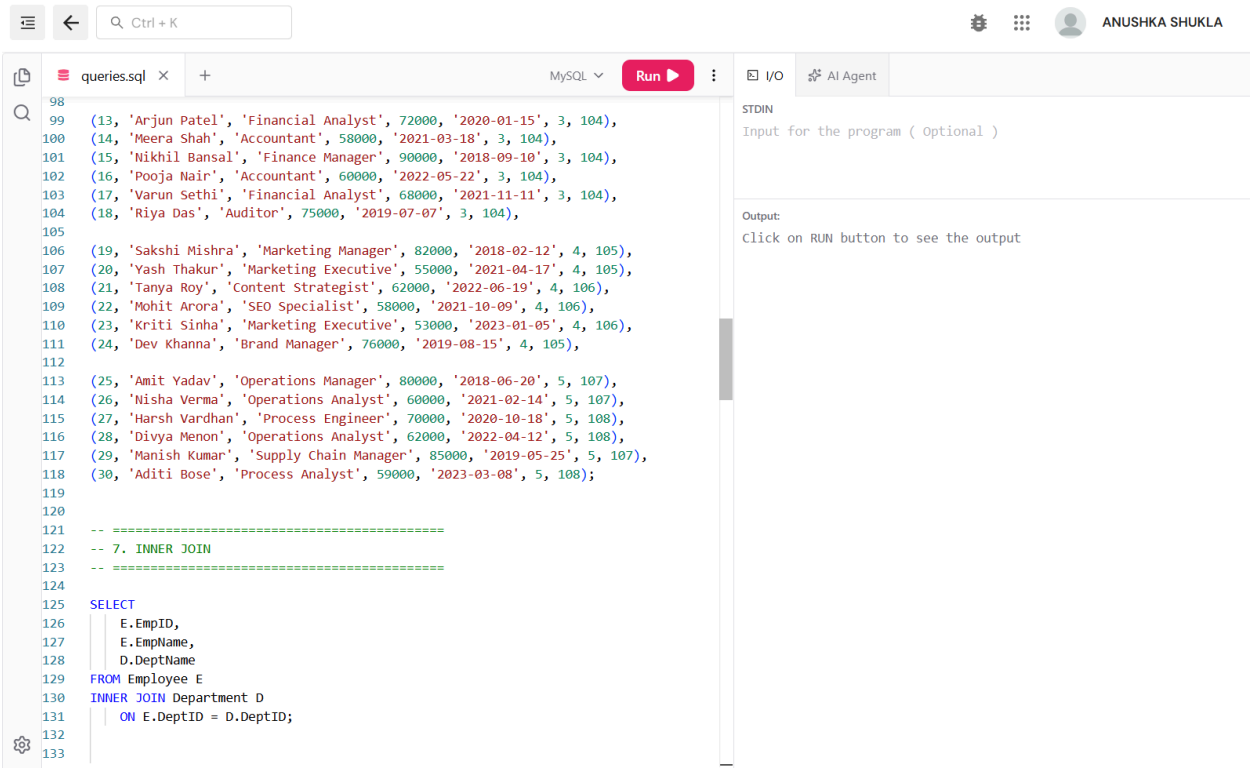
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    JobTitle VARCHAR(50),
    Salary DECIMAL(10,2),
    HireDate DATE,
    DeptID INT,
    ProjectID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID),
    FOREIGN KEY (ProjectID) REFERENCES Project(ProjectID)
);

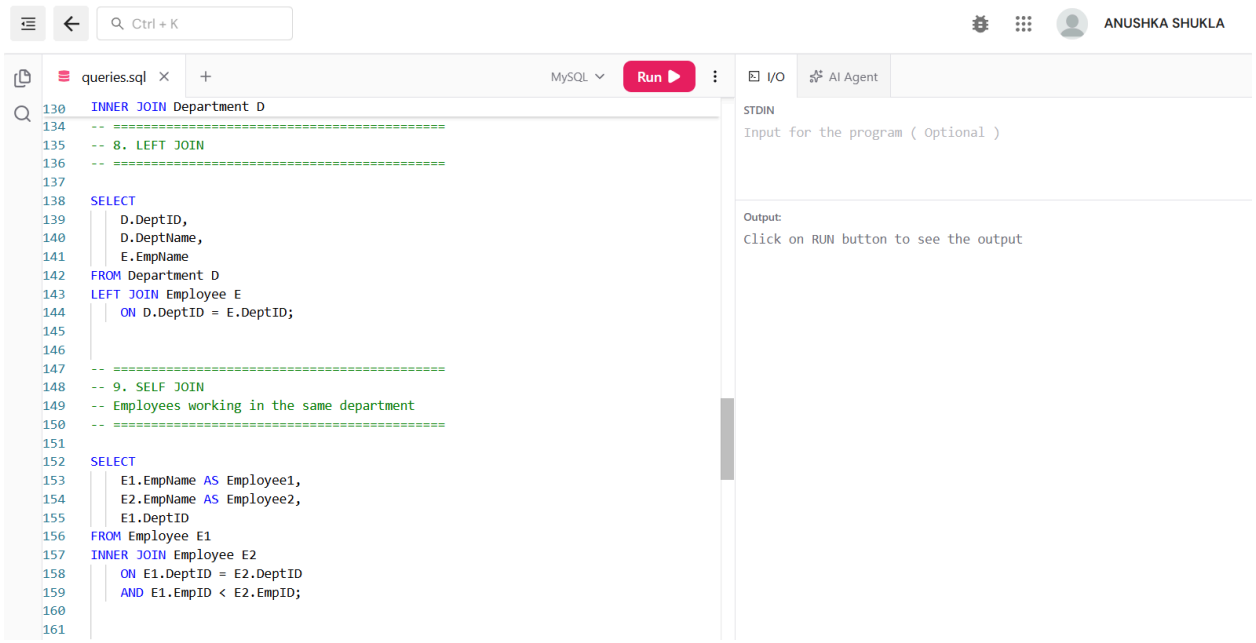

-- ============================================
-- 4. INSERT DEPARTMENT DATA
-- ============================================

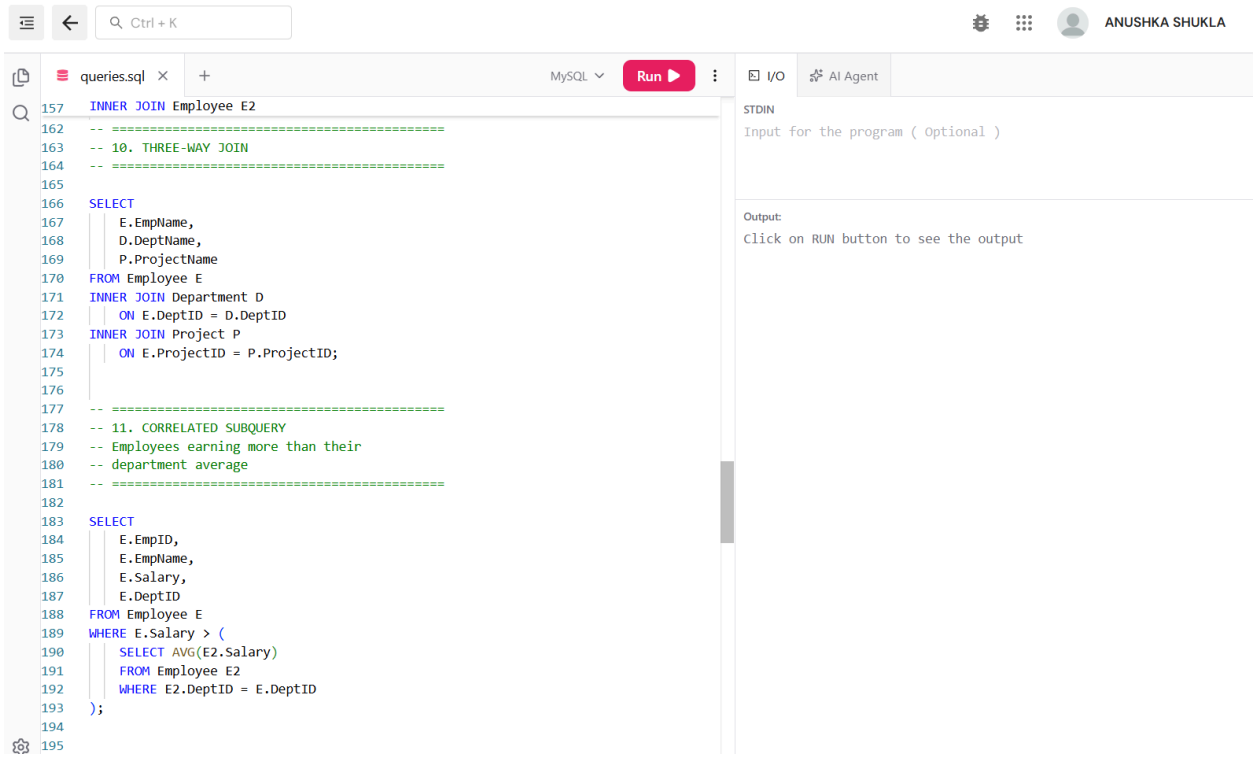
INSERT INTO Department VALUES
(1, 'IT', 'Delhi'),
(2, 'HR', 'Mumbai'),
(3, 'Finance', 'Bangalore'),
(4, 'Marketing', 'Pune'),
(5, 'Operations', 'Hyderabad');

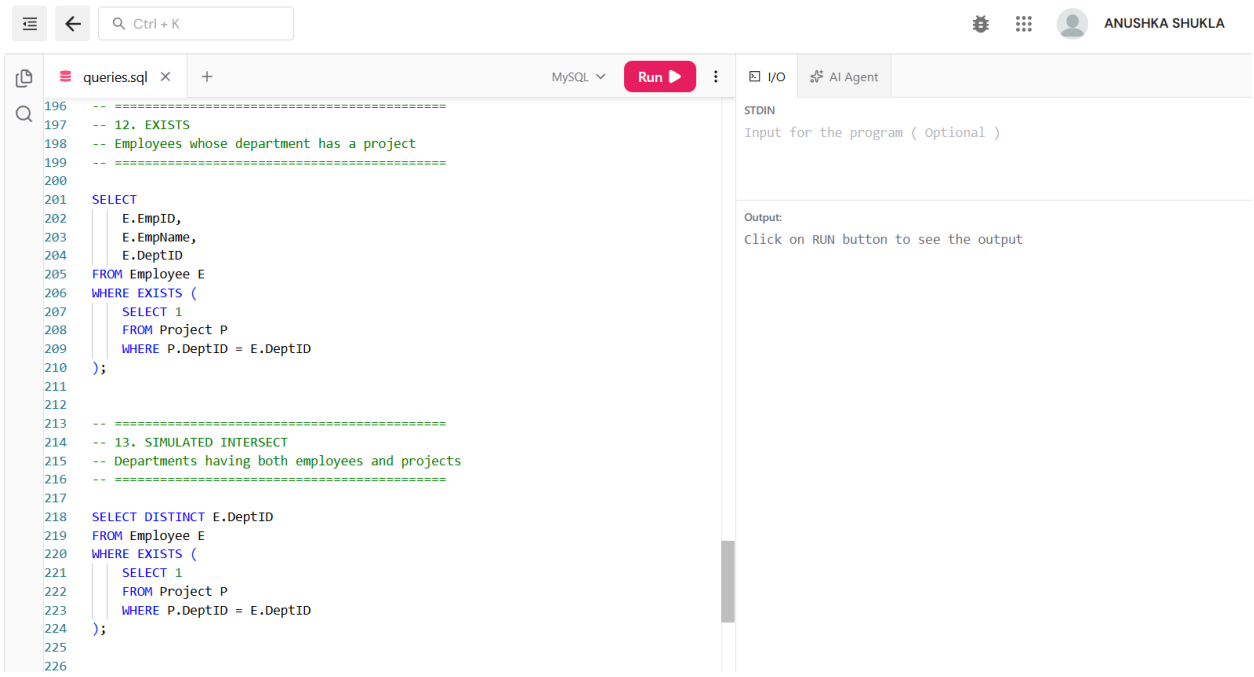
```

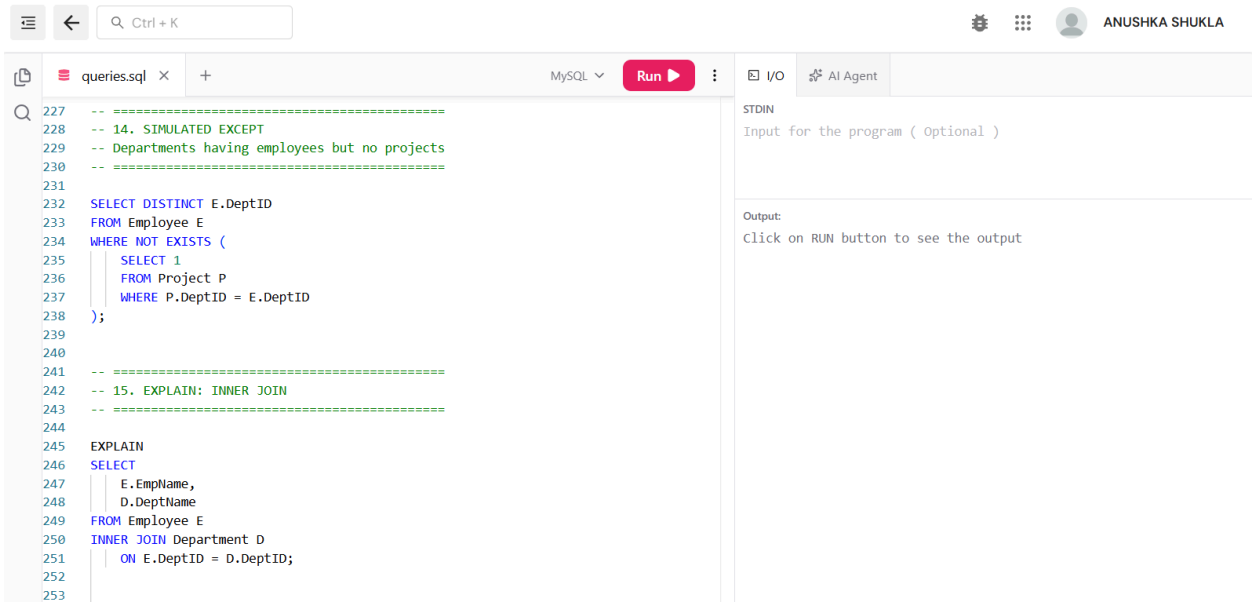

Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
-- ==========
-- 5. INSERT PROJECT DATA
-- ==========
INSERT INTO Project VALUES
(101, 'Cloud Migration', 1500000, 1),
(102, 'AI Analytics', 2000000, 1),
(103, 'Employee Portal', 800000, 2),
(104, 'Financial Audit', 1200000, 3),
(105, 'Market Research', 900000, 4),
(106, 'Digital Campaign', 1100000, 4),
(107, 'Supply Chain', 1800000, 5),
(108, 'Process Automation', 1300000, 5);
-- ==========
-- 6. INSERT EMPLOYEE DATA
-- ==========
INSERT INTO Employee VALUES
(1, 'Aarav Sharma', 'Software Engineer', 75000, '2021-01-15', 1, 101),
(2, 'Ananya Singh', 'Data Analyst', 68000, '2022-03-20', 1, 102),
(3, 'Rohan Verma', 'Cloud Engineer', 85000, '2020-07-10', 1, 101),
(4, 'Priya Gupta', 'AI Engineer', 92000, '2019-11-05', 1, 102),
(5, 'Karan Mehta', 'Software Engineer', 78000, '2021-09-12', 1, 102),
(6, 'Neha Kapoor', 'System Analyst', 72000, '2022-01-18', 1, 101),
(7, 'Rahul Joshi', 'HR Manager', 70000, '2018-05-10', 2, 103),
(8, 'Sneha Agarwal', 'HR Executive', 52000, '2021-06-15', 2, 103),
(9, 'Vikas Malhotra', 'Recruiter', 48000, '2022-08-20', 2, 103),
(10, 'Ishita Jain', 'HR Executive', 55000, '2020-12-01', 2, 103),
(11, 'Aditya Rao', 'Training Manager', 65000, '2019-04-25', 2, 103),
(12, 'Simran Kaur', 'HR Executive', 51000, '2023-02-10', 2, 103),

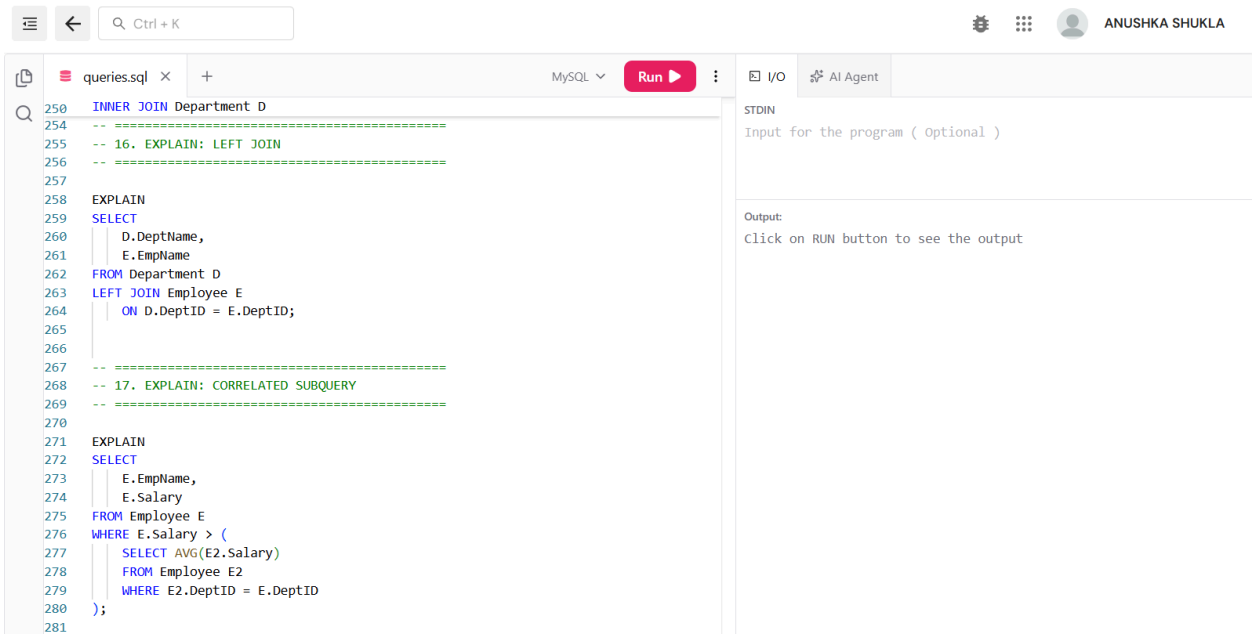

Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
(13, 'Arjun Patel', 'Financial Analyst', 72000, '2020-01-15', 3, 104),
(14, 'Meera Shah', 'Accountant', 58000, '2021-03-18', 3, 104),
(15, 'Nikhil Bansal', 'Finance Manager', 90000, '2018-09-10', 3, 104),
(16, 'Pooja Nair', 'Accountant', 60000, '2022-05-22', 3, 104),
(17, 'Varun Sethi', 'Financial Analyst', 68000, '2021-11-11', 3, 104),
(18, 'Riya Das', 'Auditor', 75000, '2019-07-07', 3, 104),
(19, 'Sakshi Mishra', 'Marketing Manager', 82000, '2018-02-12', 4, 105),
(20, 'Yash Thakur', 'Marketing Executive', 55000, '2021-04-17', 4, 105),
(21, 'Tanya Roy', 'Content Strategist', 62000, '2022-06-19', 4, 106),
(22, 'Mohit Arora', 'SEO Specialist', 58000, '2021-10-09', 4, 106),
(23, 'Kriti Sinha', 'Marketing Executive', 53000, '2023-01-05', 4, 106),
(24, 'Dev Khanna', 'Brand Manager', 76000, '2019-08-15', 4, 105),
(25, 'Amit Yadav', 'Operations Manager', 80000, '2018-06-20', 5, 107),
(26, 'Nisha Verma', 'Operations Analyst', 60000, '2021-02-14', 5, 107),
(27, 'Harsh Vardhan', 'Process Engineer', 70000, '2020-10-18', 5, 108),
(28, 'Divya Menon', 'Operations Analyst', 62000, '2022-04-12', 5, 108),
(29, 'Manish Kumar', 'Supply Chain Manager', 85000, '2019-05-25', 5, 107),
(30, 'Aditi Bose', 'Process Analyst', 59000, '2023-03-08', 5, 108);
-- ==========
-- 7. INNER JOIN
-- ==========
SELECT
E.EmpID,
E.EmpName,
D.DeptName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;

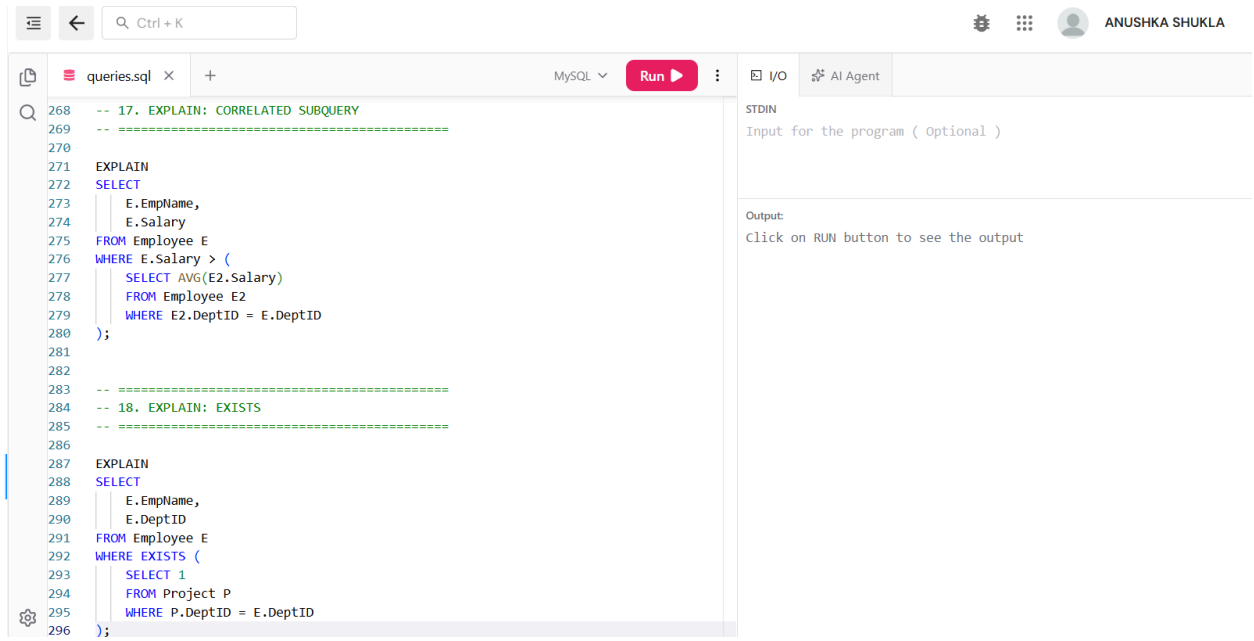

Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
INNER JOIN Department D
-- ==========
-- 8. LEFT JOIN
-- ==========
SELECT
D.DeptID,
D.DeptName,
E.EmpName
FROM Department D
LEFT JOIN Employee E
ON D.DeptID = E.DeptID;
-- ==========
-- 9. SELF JOIN
-- Employees working in the same department
-- ==========
SELECT
E1.EmpName AS Employee1,
E2.EmpName AS Employee2,
E1.DeptID
FROM Employee E1
INNER JOIN Employee E2
ON E1.DeptID = E2.DeptID
AND E1.EmpID < E2.EmpID;


Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
INNER JOIN Employee E2
-- ==========
-- 10. THREE-WAY JOIN
-- ==========
SELECT
E.EmpName,
D.DeptName,
P.ProjectName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID
INNER JOIN Project P
ON E.ProjectID = P.ProjectID;
-- ==========
-- 11. CORRELATED SUBQUERY
-- Employees earning more than their
-- department average
-- ==========
SELECT
E.EmpID,
E.EmpName,
E.Salary,
E.DeptID
FROM Employee E
WHERE E.Salary > (
SELECT AVG(E2.Salary)
FROM Employee E2
WHERE E2.DeptID = E.DeptID
);

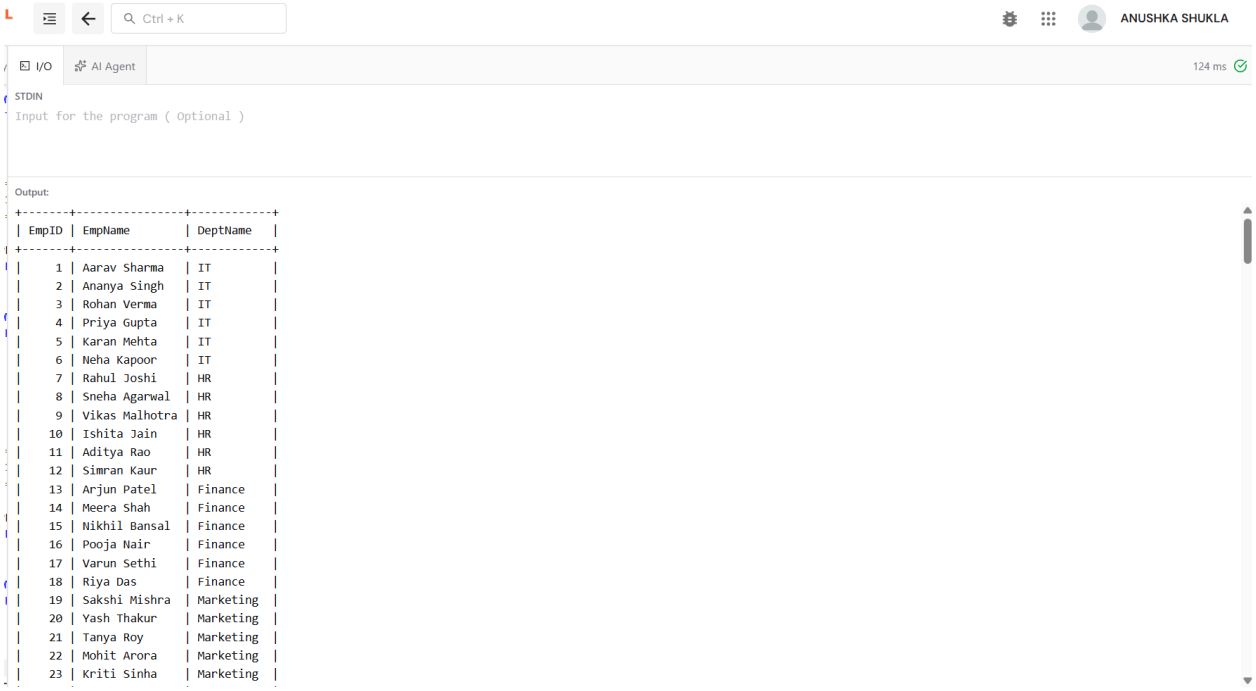

Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
-- ==========
-- 12. EXISTS
-- Employees whose department has a project
-- ==========
SELECT
E.EmpID,
E.EmpName,
E.DeptID
FROM Employee E
WHERE EXISTS (
SELECT 1
FROM Project P
WHERE P.DeptID = E.DeptID
);
-- ==========
-- 13. SIMULATED INTERSECT
-- Departments having both employees and projects
-- ==========
SELECT DISTINCT E.DeptID
FROM Employee E
WHERE EXISTS (
SELECT 1
FROM Project P
WHERE P.DeptID = E.DeptID
);

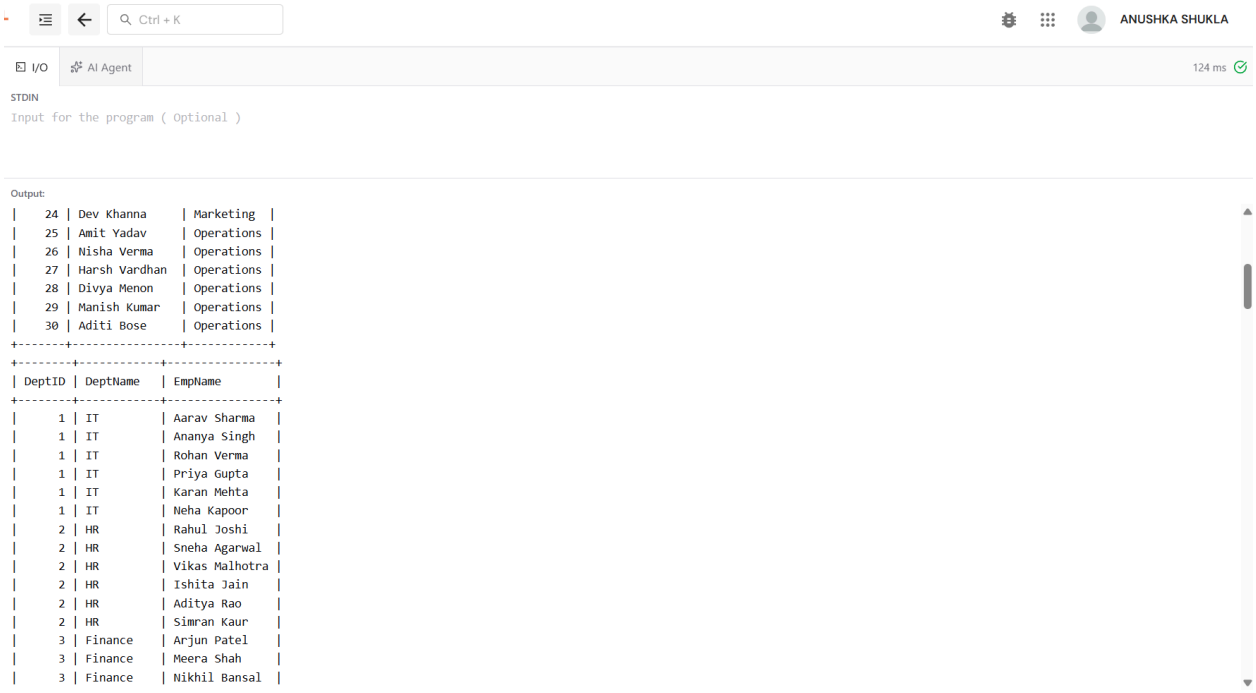

Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
-- ==========
-- 14. SIMULATED EXCEPT
-- Departments having employees but no projects
-- ==========
SELECT DISTINCT E.DeptID
FROM Employee E
WHERE NOT EXISTS (
SELECT 1
FROM Project P
WHERE P.DeptID = E.DeptID
);
-- ==========
-- 15. EXPLAIN: INNER JOIN
-- ==========
EXPLAIN
SELECT
E.EmpName,
D.DeptName
FROM Employee E
INNER JOIN Department D
ON E.DeptID = D.DeptID;

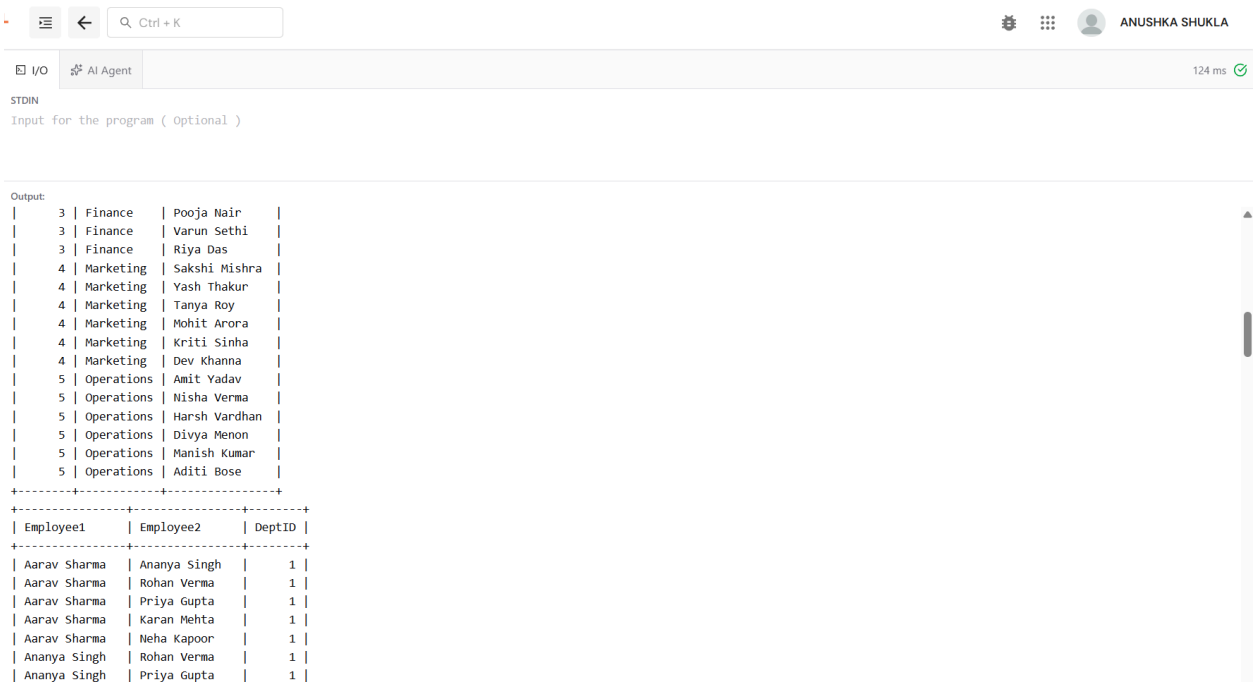

Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
250 INNER JOIN Department D
254 -- ==
255 -- 16. EXPLAIN: LEFT JOIN
256 -- ==
257
258 EXPLAIN
259 SELECT
260 D.DeptName,
261 E.EmpName
262 FROM Department D
263 LEFT JOIN Employee E
264 ON D.DeptID = E.DeptID;
265
266
267 -- ==
268 -- 17. EXPLAIN: CORRELATED SUBQUERY
269 -- ==
270
271 EXPLAIN
272 SELECT
273 E.EmpName,
274 E.Salary
275 FROM Employee E
276 WHERE E.Salary > (
277 SELECT AVG(E2.Salary)
278 FROM Employee E2
279 WHERE E2.DeptID = E.DeptID
280);
281

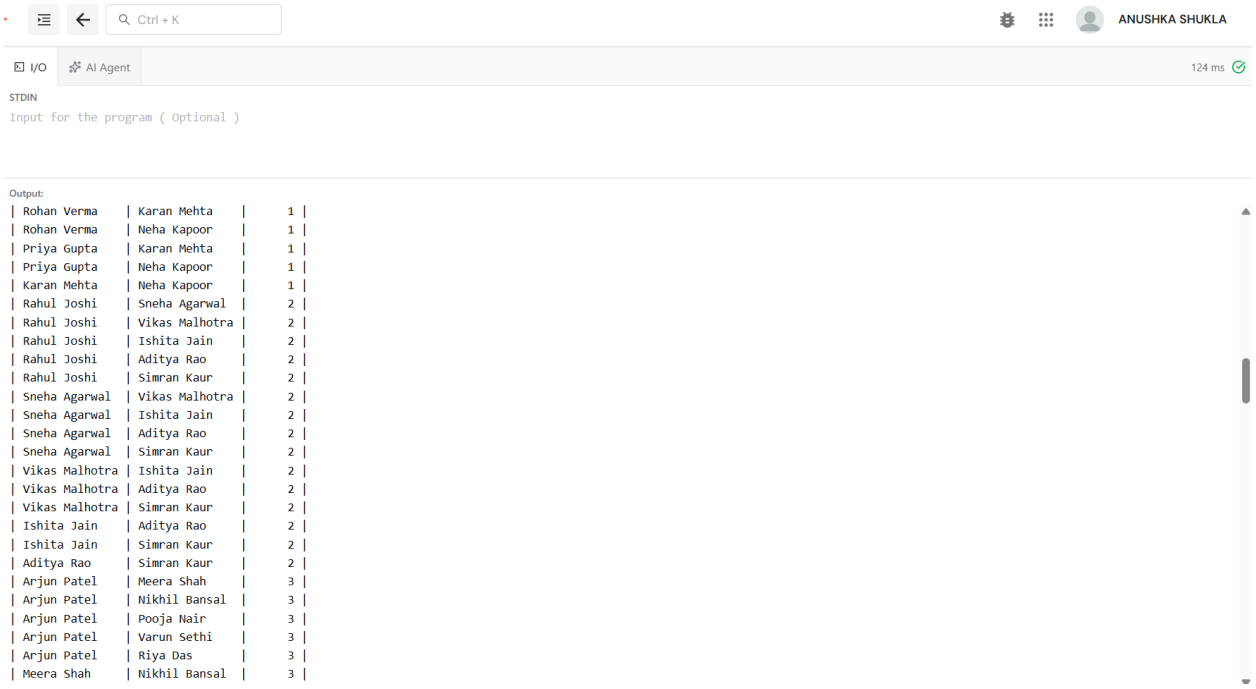

Ctrl + K
ANUSHKA SHUKLA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
268 -- 17. EXPLAIN: CORRELATED SUBQUERY
269 -- ==
270
271 EXPLAIN
272 SELECT
273 E.EmpName,
274 E.Salary
275 FROM Employee E
276 WHERE E.Salary > (
277 SELECT AVG(E2.Salary)
278 FROM Employee E2
279 WHERE E2.DeptID = E.DeptID
280);
281
282
283 -- ==
284 -- 18. EXPLAIN: EXISTS
285 -- ==
286
287 EXPLAIN
288 SELECT
289 E.EmpName,
290 E.DeptID
291 FROM Employee E
292 WHERE EXISTS (
293 SELECT 1
294 FROM Project P
295 WHERE P.DeptID = E.DeptID
296);

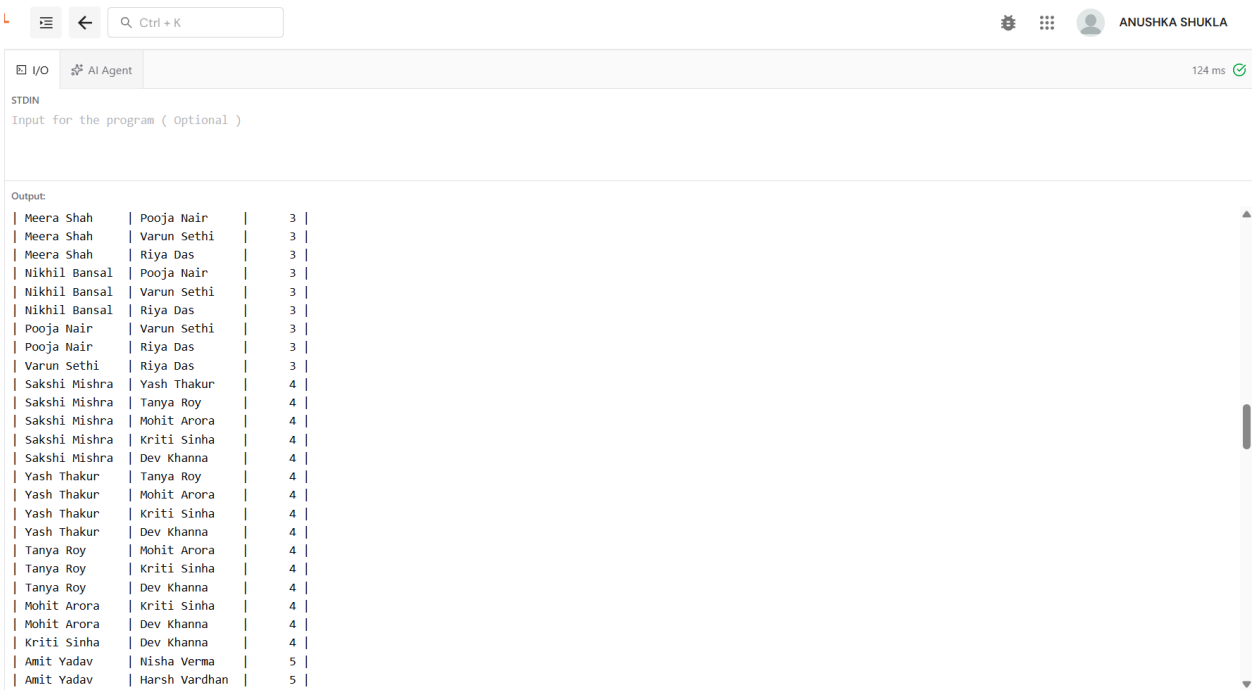
OUTPUT

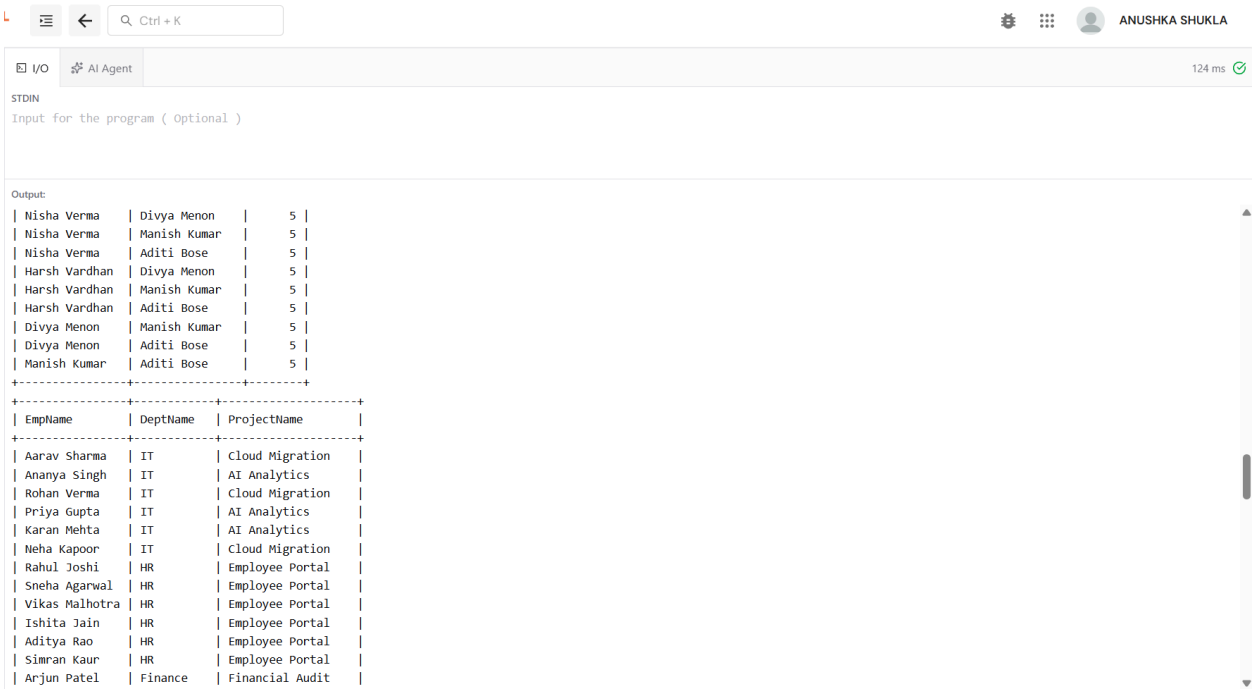

Ctrl + K
ANUSHKA SHUKLA
I/O
AI Agent
124 ms
STDIN
Input for the program (Optional)
Output:
EmpID	EmpName	DeptName
1	Aarav Sharma	IT
2	Ananya Singh	IT
3	Rohan Verma	IT
4	Priya Gupta	IT
5	Karan Mehta	IT
6	Neha Kapoor	IT
7	Rahul Joshi	HR
8	Sneha Agarwal	HR
9	Vikas Malhotra	HR
10	Ishita Jain	HR
11	Aditya Rao	HR
12	Simran Kaur	HR
13	Arjun Patel	Finance
14	Meera Shah	Finance
15	Nikhil Bansal	Finance
16	Pooja Nair	Finance
17	Varun Sethi	Finance
18	Riya Das	Finance
19	Sakshi Mishra	Marketing
20	Yash Thakur	Marketing
21	Tanya Roy	Marketing
22	Mohit Arora	Marketing
23	Kriti Sinha	Marketing

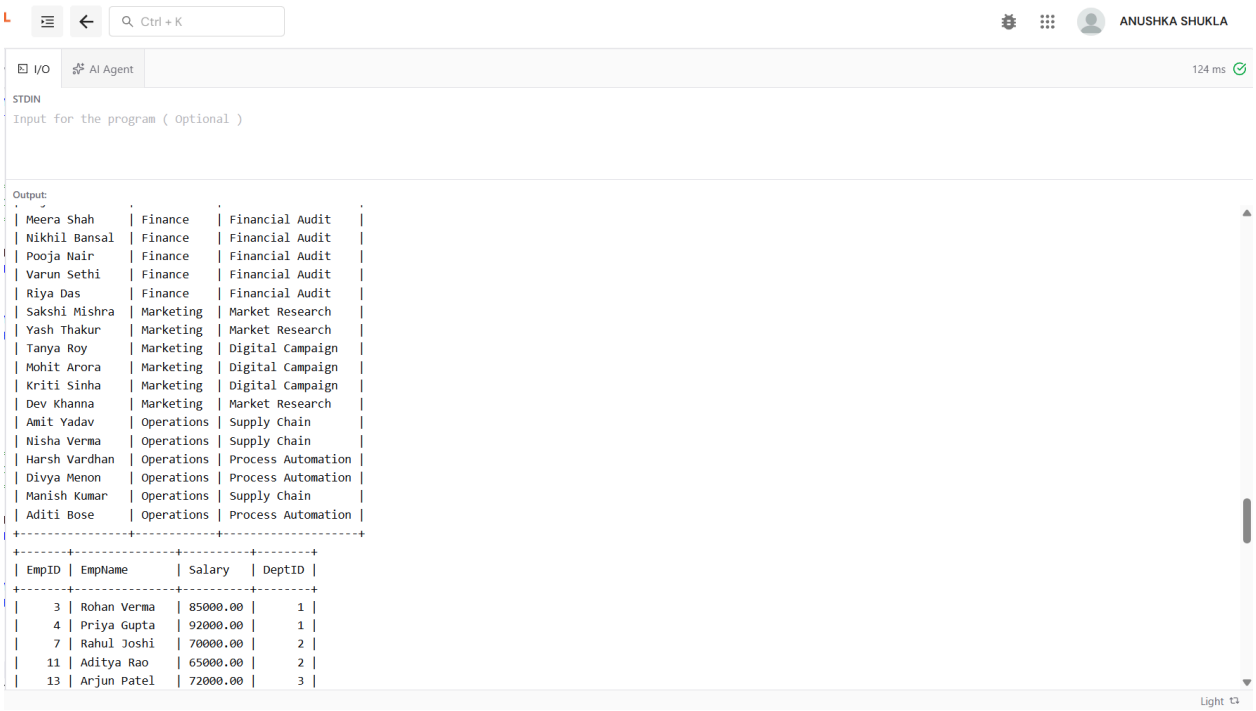

Ctrl + K
ANUSHKA SHUKLA
I/O
AI Agent
124 ms
STDIN
Input for the program (Optional)
Output:
24	Dev Khanna	Marketing
25	Amit Yadav	Operations
26	Nisha Verma	Operations
27	Harsh Vardhan	Operations
28	Divya Menon	Operations
29	Manish Kumar	Operations
30	Aditi Bose	Operations
DeptID	DeptName	EmpName
1	IT	Aarav Sharma
1	IT	Ananya Singh
1	IT	Rohan Verma
1	IT	Priya Gupta
1	IT	Karan Mehta
1	IT	Neha Kapoor
2	HR	Rahul Joshi
2	HR	Sneha Agarwal
2	HR	Vikas Malhotra
2	HR	Ishita Jain
2	HR	Aditya Rao
2	HR	Simran Kaur
3	Finance	Arjun Patel
3	Finance	Meera Shah
3	Finance	Nikhil Bansal

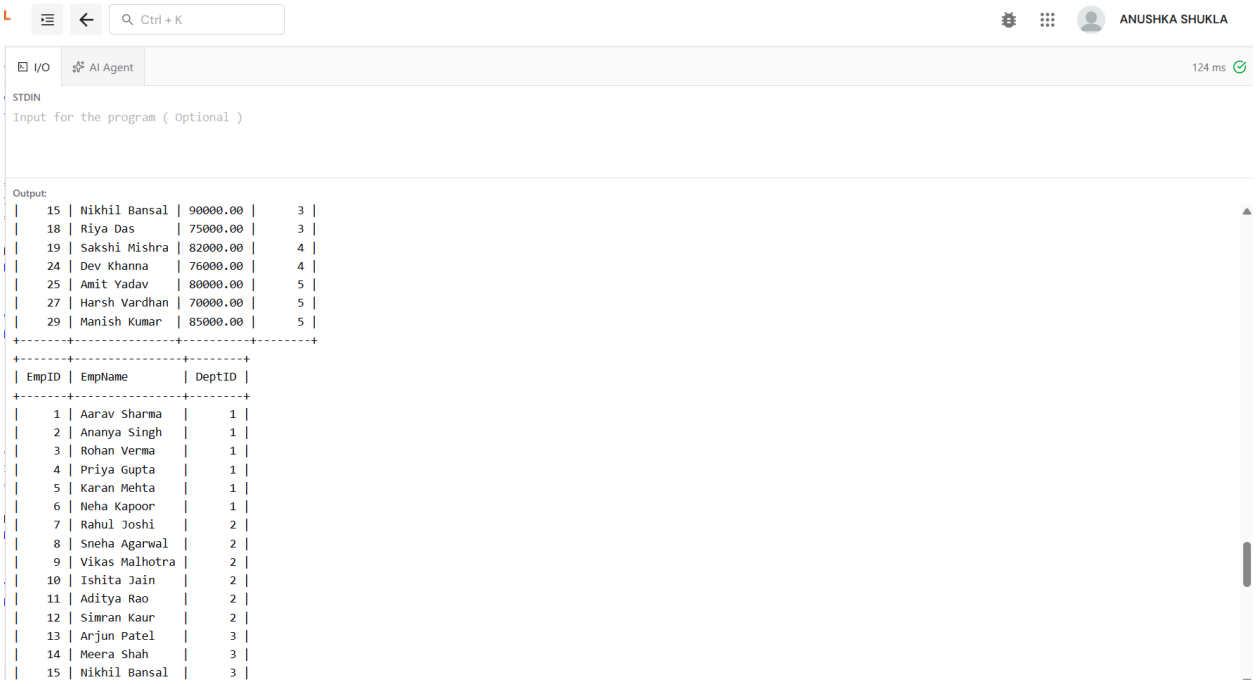

Ctrl + K
ANUSHKA SHUKLA
I/O
AI Agent
124 ms
STDIN
Input for the program (Optional)
Output:
3	Finance	Pooja Nair
3	Finance	Varun Sethi
3	Finance	Riya Das
4	Marketing	Sakshi Mishra
4	Marketing	Yash Thakur
4	Marketing	Tanya Roy
4	Marketing	Mohit Arora
4	Marketing	Kriti Sinha
4	Marketing	Dev Khanna
5	Operations	Amit Yadav
5	Operations	Nisha Verma
5	Operations	Harsh Vardhan
5	Operations	Divya Menon
5	Operations	Manish Kumar
5	Operations	Aditi Bose
+--------+------------+---------------+
+---------------+---------------+--------+
| Employee1 | Employee2 | DeptID |
+---------------+---------------+--------+
Aarav Sharma	Ananya Singh	1
Aarav Sharma	Rohan Verma	1
Aarav Sharma	Priya Gupta	1
Aarav Sharma	Karan Mehta	1
Aarav Sharma	Neha Kapoor	1
Ananya Singh	Rohan Verma	1
Ananya Singh	Priya Gupta	1

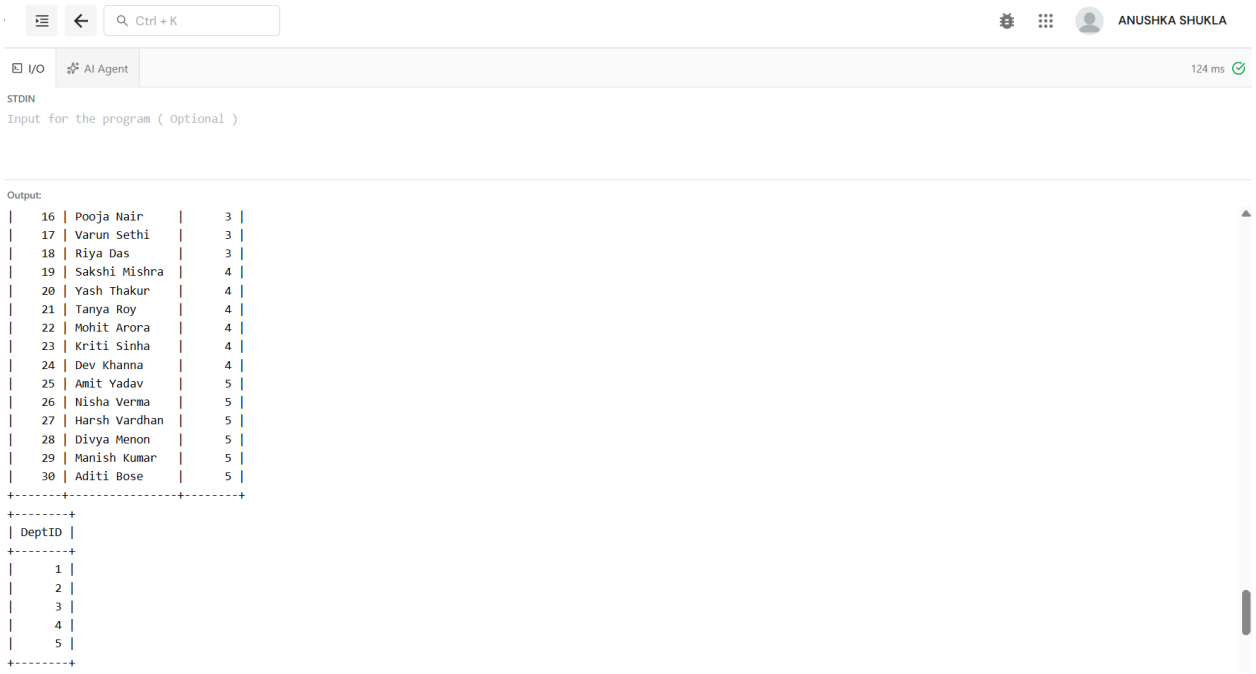

Ctrl + K
ANUSHKA SHUKLA
I/O
AI Agent
124 ms
STDIN
Input for the program (Optional)
Output:
Rohan Verma	Karan Mehta	1
Rohan Verma	Neha Kapoor	1
Priya Gupta	Karan Mehta	1
Priya Gupta	Neha Kapoor	1
Karan Mehta	Neha Kapoor	1
Rahul Joshi	Sneha Agarwal	2
Rahul Joshi	Vikas Malhotra	2
Rahul Joshi	Ishita Jain	2
Rahul Joshi	Aditya Rao	2
Rahul Joshi	Simran Kaur	2
Sneha Agarwal	Vikas Malhotra	2
Sneha Agarwal	Ishita Jain	2
Sneha Agarwal	Aditya Rao	2
Sneha Agarwal	Simran Kaur	2
Vikas Malhotra	Ishita Jain	2
Vikas Malhotra	Aditya Rao	2
Vikas Malhotra	Simran Kaur	2
Ishita Jain	Aditya Rao	2
Ishita Jain	Simran Kaur	2
Aditya Rao	Simran Kaur	2
Arjun Patel	Meera Shah	3
Arjun Patel	Nikhil Bansal	3
Arjun Patel	Pooja Nair	3
Arjun Patel	Varun Sethi	3
Arjun Patel	Riya Das	3
Meera Shah	Nikhil Bansal	3

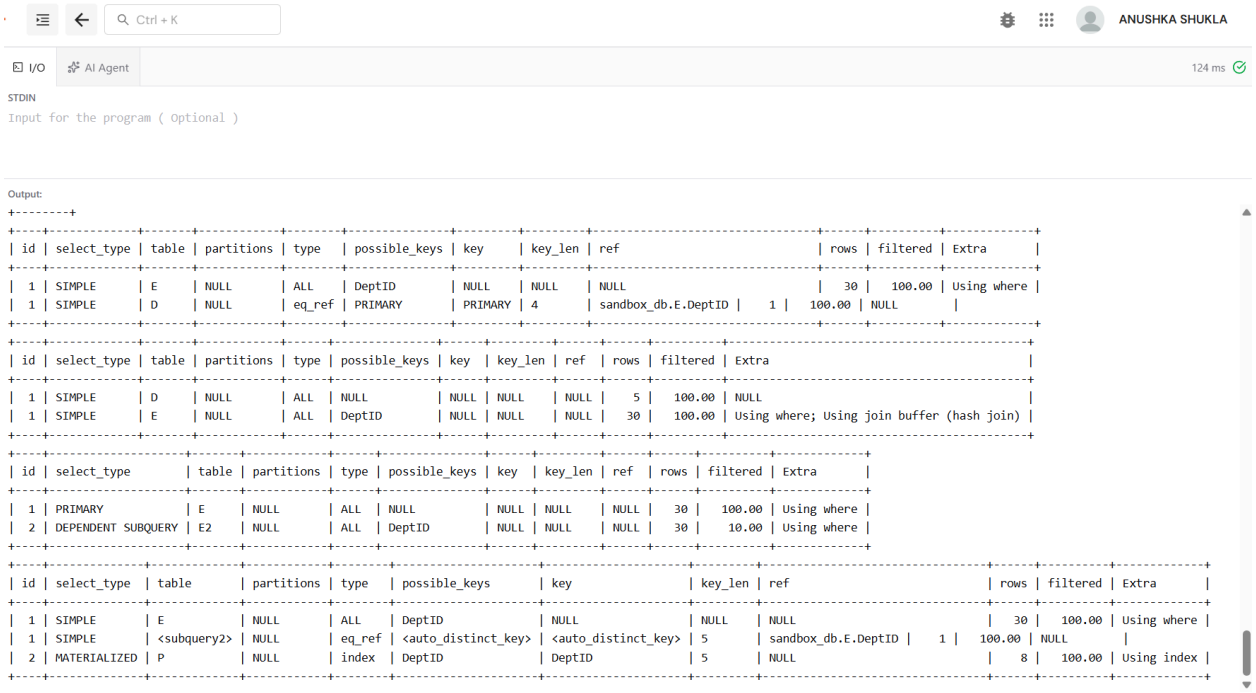

Ctrl + K
ANUSHKA SHUKLA
I/O
AI Agent
124 ms
STDIN
Input for the program (Optional)
Output:
Meera Shah	Pooja Nair	3
Meera Shah	Varun Sethi	3
Meera Shah	Riya Das	3
Nikhil Bansal	Pooja Nair	3
Nikhil Bansal	Varun Sethi	3
Nikhil Bansal	Riya Das	3
Pooja Nair	Varun Sethi	3
Pooja Nair	Riya Das	3
Varun Sethi	Riya Das	3
Sakshi Mishra	Yash Thakur	4
Sakshi Mishra	Tanya Roy	4
Sakshi Mishra	Mohit Arora	4
Sakshi Mishra	Kriti Sinha	4
Sakshi Mishra	Dev Khanna	4
Yash Thakur	Tanya Roy	4
Yash Thakur	Mohit Arora	4
Yash Thakur	Kriti Sinha	4
Yash Thakur	Dev Khanna	4
Tanya Roy	Mohit Arora	4
Tanya Roy	Kriti Sinha	4
Tanya Roy	Dev Khanna	4
Mohit Arora	Kriti Sinha	4
Mohit Arora	Dev Khanna	4
Kriti Sinha	Dev Khanna	4
Amit Yadav	Nisha Verma	5
Amit Yadav	Harsh Vardhan	5


Ctrl + K
ANUSHKA SHUKLA
I/O
AI Agent
124 ms
STDIN
Input for the program (Optional)
Output:
Nisha Verma	Divya Menon	5
Nisha Verma	Manish Kumar	5
Nisha Verma	Aditi Bose	5
Harsh Vardhan	Divya Menon	5
Harsh Vardhan	Manish Kumar	5
Harsh Vardhan	Aditi Bose	5
Divya Menon	Manish Kumar	5
Divya Menon	Aditi Bose	5
Manish Kumar	Aditi Bose	5
+----------+----------+-------+
+----------+----------+----------+
| EmpName | DeptName | ProjectName |
+----------+----------+----------+
Aarav Sharma	IT	Cloud Migration
Ananya Singh	IT	AI Analytics
Rohan Verma	IT	Cloud Migration
Priya Gupta	IT	AI Analytics
Karan Mehta	IT	AI Analytics
Neha Kapoor	IT	Cloud Migration
Rahul Joshi	HR	Employee Portal
Sneha Agarwal	HR	Employee Portal
Vikas Malhotra	HR	Employee Portal
Ishita Jain	HR	Employee Portal
Aditya Rao	HR	Employee Portal
Simran Kaur	HR	Employee Portal
Arjun Patel	Finance	Financial Audit


Ctrl + K
ANUSHKA SHUKLA
I/O
AI Agent
124 ms
STDIN
Input for the program (Optional)
Output:
Meera Shah	Finance	Financial Audit
Nikhil Bansal	Finance	Financial Audit
Pooja Nair	Finance	Financial Audit
Varun Sethi	Finance	Financial Audit
Riya Das	Finance	Financial Audit
Sakshi Mishra	Marketing	Market Research
Yash Thakur	Marketing	Market Research
Tanya Roy	Marketing	Digital Campaign
Mohit Arora	Marketing	Digital Campaign
Kriti Sinha	Marketing	Digital Campaign
Dev Khanna	Marketing	Market Research
Amit Yadav	Operations	Supply Chain
Nisha Verma	Operations	Supply Chain
Harsh Vardhan	Operations	Process Automation
Divya Menon	Operations	Process Automation
Manish Kumar	Operations	Supply Chain
Aditi Bose	Operations	Process Automation
+---+---+---+---+
| EmpID | EmpName | Salary | DeptID |
+---+---+---+---+
3	Rohan Verma	85000.00	1
4	Priya Gupta	92000.00	1
7	Rahul Joshi	70000.00	2
11	Aditya Rao	65000.00	2
13	Arjun Patel	72000.00	3
Light


Ctrl + K
ANUSHKA SHUKLA
I/O
AI Agent
124 ms
STDIN
Input for the program (Optional)
Output:
15	Nikhil Bansal	90000.00	3
18	Riya Das	75000.00	3
19	Sakshi Mishra	82000.00	4
24	Dev Khanna	76000.00	4
25	Amit Yadav	80000.00	5
27	Harsh Vardhan	70000.00	5
29	Manish Kumar	85000.00	5
+---+---+---+---+
| EmpID | EmpName | DeptID |
+---+---+---+
1	Aarav Sharma	1
2	Ananya Singh	1
3	Rohan Verma	1
4	Priya Gupta	1
5	Karan Mehta	1
6	Neha Kapoor	1
7	Rahul Joshi	2
8	Sneha Agarwal	2
9	Vikas Malhotra	2
10	Ishita Jain	2
11	Aditya Rao	2
12	Simran Kaur	2
13	Arjun Patel	3
14	Meera Shah	3
15	Nikhil Bansal	3


Ctrl + K
ANUSHKA SHUKLA
I/O
AI Agent
124 ms
STDIN
Input for the program (Optional)
Output:
16	Pooja Nair	3
17	Varun Sethi	3
18	Riya Das	3
19	Sakshi Mishra	4
20	Yash Thakur	4
21	Tanya Roy	4
22	Mohit Arora	4
23	Kriti Sinha	4
24	Dev Khanna	4
25	Amit Yadav	5
26	Nisha Verma	5
27	Harsh Vardhan	5
28	Divya Menon	5
29	Manish Kumar	5
30	Aditi Bose	5
DeptID
1
2
3
4
5


Ctrl + K
ANUSHKA SHUKLA
I/O
AI Agent
124 ms
STDIN
Input for the program (Optional)
Output:
id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra
1 | SIMPLE | E | NULL | ALL | DeptID | NULL | NULL | NULL | 30 | 100.00 | Using where
1 | SIMPLE | D | NULL | eq_ref | PRIMARY | PRIMARY | 4 | sandbox_db.E.DeptID | 1 | 100.00 | NULL
id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra
1 | SIMPLE | D | NULL | ALL | NULL | NULL | NULL | NULL | 5 | 100.00 | NULL
1 | SIMPLE | E | NULL | ALL | DeptID | NULL | NULL | NULL | 30 | 100.00 | Using where; Using join buffer (hash join)
id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra
1 | PRIMARY | E | NULL | ALL | NULL | NULL | NULL | NULL | 30 | 100.00 | Using where
2 | DEPENDENT SUBQUERY | E2 | NULL | ALL | DeptID | NULL | NULL | NULL | 30 | 10.00 | Using where
id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra
1 | SIMPLE | E | NULL | ALL | DeptID | NULL | NULL | NULL | 30 | 100.00 | Using where
1 | SIMPLE | <subquery2> | NULL | eq_ref | <auto_distinct_key> | <auto_distinct_key> | 5 | sandbox_db.E.DeptID | 1 | 100.00 | NULL
2 | MATERIALIZED | P | NULL | index | DeptID | DeptID | 5 | NULL | 8 | 100.00 | Using index

OBSERVATIONS/OUTCOME

1. Employee, Department, and Project tables were successfully created and populated with the required data.
2. INNER JOIN successfully retrieved matching employee and department records.
3. LEFT JOIN successfully displayed all departments along with their corresponding employees.
4. A self-join successfully identified employees belonging to the same department.
5. A 3-way join successfully retrieved employee, department, and project details together.
6. The correlated subquery successfully identified employees whose salary was greater than the average salary of their respective department.
7. The EXISTS operator successfully checked for the existence of projects associated with employees' departments.
8. INTERSECT and EXCEPT operations were successfully simulated using EXISTS and NOT EXISTS.
9. EXPLAIN successfully displayed the execution plans for different SQL queries, allowing their execution strategies to be examined and compared.